Model Training

1. Training and Test Datasets

When building machine learning models, one of the most important steps is to split the dataset into training and test sets. This practice ensures that the model is trained on one part of the data and evaluated on a separate, unseen part. The primary goal of splitting the data is to assess how well the model generalizes to new, unseen data (i.e., data that it wasn't trained on).

- Prevent Overfitting: By testing the model on a separate test set, we can check whether it is overfitting to the training data, which occurs when the model learns the noise or details specific to the training set but performs poorly on new data.
- Evaluate Model Performance: A test set provides an unbiased estimate of how well the model will perform on unseen data. Without a test set, we can't reliably assess the model's performance on real-world data.
- Avoid Data Leakage: If the same data is used for both training and testing, the model may simply memorize the data (data leakage), which results in unrealistic performance metrics and poor generalization.

1.1 Train-Test Split

The most basic approach to dataset splitting is the train-test split, which divides the data into two distinct parts:

- **Training Set**: The model is trained (or fitted) using the training set. The model learns the relationships between the input features and the target variable in this set.
- **Test Set**: The test set is used to evaluate the model's performance after training and provides an estimate of how well the model will perform on new data. The model does not see this data during training.

Typical Split Ratios:

- **80/20 Split**: 80% of the data for training and 20% for testing.
- **70/30 Split**: 70% of the data for training and 30% for testing.

1.2 Split a Dataset scikit-learn

The scikit-learn Python machine learning library provides an implementation of the train-test split evaluation procedure via the train_test_split() function. The function takes a loaded dataset as input and returns the dataset split into two subsets.

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv('winequality-red.csv')
train, test = train_test_split(df, test_size=0.2)
```

test_size=0.2 Specifies that 20% of the data will be used for testing and 80% for training.

The data will be split randomly to ensure that both the training and test sets are representative of the overall dataset. A non-random split can introduce bias, causing the test set to differ significantly from the training set, which will affect the model's performance evaluation. If all the training data comes from one specific group (e.g., a specific time period or location), the model may not generalize well to the test data. Random splitting ensures that both sets cover the full range of the data distribution.

1.3 Cross-Validation

Instead of performing a single train-test split, we can use cross-validation to get a more reliable estimate of model performance. In k-fold cross-validation, the dataset is split into k equal parts (or "folds"). The model is trained on k-1 folds and tested on the remaining fold. This process is repeated k times, with a different fold used as the test set each time.

Advantages of cross-validation:

- Reduces variance in performance estimates, as the model is evaluated on multiple test sets.
- More reliable when dealing with small datasets.

If the dataset is large enough, a simple 80/20 or 70/30 split is usually sufficient. If you have a small dataset, using cross-validation may provide more reliable performance estimates.

2. Hyperparameters

In machine learning, **hyperparameters** are the parameters that define the model's behavior and learning process, but they are not learned from the data itself. Instead, they are set *before* the training process begins and control how the model is trained. Examples include the learning rate in gradient descent, the depth of a decision tree, or the number of hidden layers in a neural network.

The choice of hyperparameters can significantly impact the performance of a machine learning model. Poorly chosen hyperparameters may lead to:

- **Underfitting**: If the model is too simple (e.g., decision tree depth is too shallow), it may fail to capture important patterns in the data.

- **Overfitting**: If the model is too complex (e.g., decision tree depth is too deep), it may capture noise in the training data and perform poorly on unseen data.

Proper tuning of hyperparameters helps balance this trade-off and improve the model's generalization ability, which is why hyperparameter tuning is an essential part of building effective machine learning models.

3. Red Wine Quality Model Example

Now, we'll explore how to apply machine learning techniques to predict the quality of red wine using the famous **Red Wine Quality dataset**. The dataset consists of various chemical tests of wines, and the target variable is the quality score (from 0 to 10).

The Red Wine Quality dataset can be downloaded from Moodle. It is taken from UCI Machine Learning Repository. It contains the following attributes:

- Fixed acidity
- Volatile acidity
- Citric acid
- Residual sugar
- Chlorides
- Free sulfur dioxide
- Total sulfur dioxide
- Density
- pH
- Sulphates
- Alcohol
- Quality (Target variable)

4.1 Loading and Understanding the Data

Start by importing the necessary libraries and loading the dataset.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('winequality-red.csv')

# Basic information about the dataset
print(df.info())
print(df.describe())
```

Seaborn is a library for making statistical graphics in Python. We'll use it to visualize the data.

4.2 Exploratory Data Analysis (EDA)

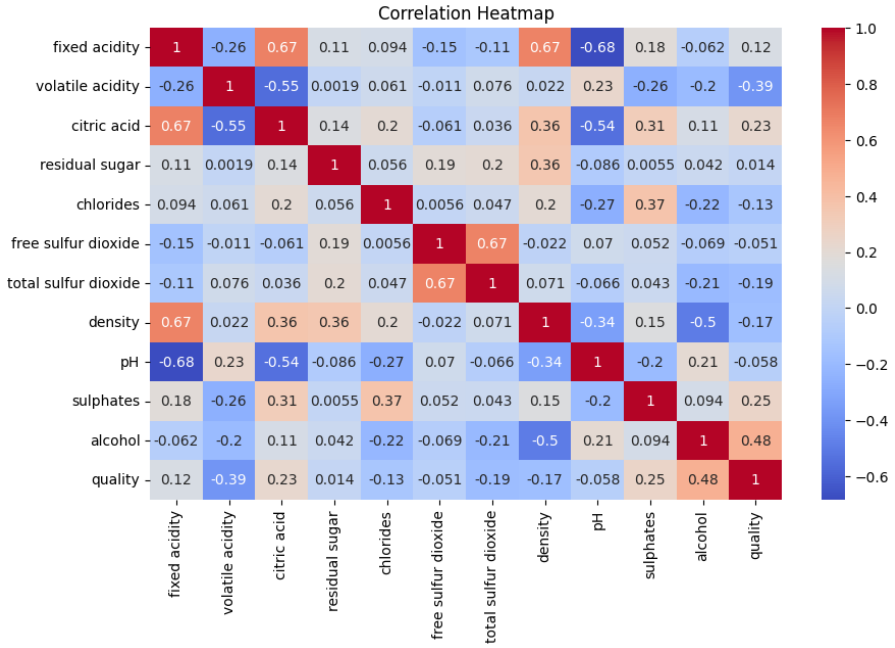
Let's explore the data to understand relationships between variables. First let's check for missing values.

```
print(df.isnull().sum())
```

There are no missing values. We can then visualize the correlation between features using a heatmap.

```
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

Note how features like alcohol, sulphates, and citric acid have a positive relationship with quality.



4.3 Data Preprocessing

Before modeling, we need to preprocess the data. Since the quality score ranges between 0 to 10, you can create a binary classification problem (good/bad wine) by using a threshold value. Let's assume the wine is good if the quality is 7 or more.

```
df['quality'] = df['quality'].apply(lambda x: 1 if x >= 7 else 0)
```

Let's check how many records we have for quality 0 and 1.

```
df['quality'].value_counts()
```

```
quality
0    1382
1     217
```

We have about six times more of class 0 (bad wine) compared to class 1 (good wine). So, the class is heavily imbalanced. Resampling can help ensure the model doesn't bias towards the majority class. There are several ways to handle this. We will use Oversampling in this example. It increases the number of instances in the minority class by creating synthetic samples or by duplicating existing samples.

First let's define the features and target and then apply SMOTE (Synthetic Minority Over-sampling Technique).

```
from imblearn.over_sampling import SMOTE
X = df.drop('quality', axis=1)
y = df['quality']
smote = SMOTE() # SMOTE (Synthetic Minority Over-sampling Technique)
X_resampled, y_resampled = smote.fit_resample(X, y)
```

```
# Check the new class distribution
print(pd.Series(y_resampled).value_counts())
```

Now we have similar number of cases for both negative and positive classes. Then we can split the data in to training and test set.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2)
```

Features in a dataset can have very different scales. For example, in the Red Wine Quality dataset:

- Alcohol is typically measured in percentages (like 10-14%).
- pH is on a scale from around 3 to 4.
- Residual sugar may range from 0 to 15 g/L.

Machine learning algorithms may disproportionately prioritize features with larger scales, such as residual sugar, over smaller-scale features like pH. So, we should Standardize the data to ensure that all features contribute equally to the model.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

4.4 Model Building

We'll now train a Logistic Regression model. We'll use the training data set to train the model and then use the test dataset to predict values. Based on the predicted values for the test dataset, we calculate the accuracy and confusion matrix.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Logistic Regression Model
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

# Predictions
y_pred_lr = log_reg.predict(X_test)

# Evaluate the model
print('Accuracy', accuracy_score(y_test, y_pred_lr))
print(confusion_matrix(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

This model will give you an accuracy of more than 75%. Note that the accuracy you get may vary depending on the random training-test split you have got.

Let's train another model using Decision Tree classifier.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
d_tree = DecisionTreeClassifier(criterion='entropy')
d_tree.fit(X_train, y_train)
```

```
# Predictions
y_pred_dt = d_tree.predict(X_test)

# Evaluate the model
print('Accuracy', accuracy_score(y_test, y_pred_dt))
print(confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
```

For this model you may get almost the same or a higher accuracy.

5. Activity

Try different models with different features considering the exploratory analysis you did earlier. Find out which model gives the best accuracy. Upload your notebook to Moodle.